

****ICSE CLASS 10 COMPUTER APPLICATIONS — COMPLETE REVISION NOTES****

****CHAPTER 1: REVISION OF CLASS IX SYLLABUS – JAVA PROGRAMMING BASICS****

****YouTube Lecture:**** <https://youtu.be/nQz-FcmwGH8?si=dEEAOe8snX0v19F9>

****1. Object Oriented Programming (OOP) Concepts****

Object Oriented Programming is a programming paradigm based on the concept of "objects" which contain data and code. The four main principles of OOP are:

****Encapsulation:**** The wrapping up of data (variables) and methods (functions) together into a single unit called a class. It also involves hiding the internal details and protecting data from unauthorised access.

****Inheritance:**** The mechanism by which one class acquires the properties and behaviour of another class. The class that gives its properties is the base/super/parent class. The class that takes the properties is the derived/sub/child class. For example, a class "Car" can inherit from a class "Vehicle".

****Polymorphism:**** The ability of an object to take many forms. The same method or operator can behave differently in different situations. For example, the '+' operator adds two numbers but concatenates two strings. Method overloading is an example of polymorphism.

****Abstraction:**** The act of representing essential features without including the background details. It focuses on 'what' an object does instead of 'how' it does it. For example, while driving a car, you only need to know how to use the steering wheel and pedals, not how the engine works internally.

****Difference between Procedure Oriented and Object Oriented Programming:****

In Procedure Oriented Programming, the focus is on functions and procedures. Programs are divided into small parts called functions. Data moves openly from function to function. Example: C, Fortran.

In Object Oriented Programming, the focus is on objects that encapsulate both data and functions. Programs are divided into objects. Data is hidden and cannot be accessed by external functions. Example: Java, C++.

****2. Introduction to Java****

Java is an object-oriented, platform-independent programming language developed by James Gosling at Sun Microsystems (now Oracle).

****Java Source Code:**** The program written by the programmer in a .java file.

****Byte Code:**** The compiler (javac) converts the source code into byte code (.class file). Byte code is not machine code; it is an intermediate code.

****Java Virtual Machine (JVM):**** The interpreter (java) takes the byte code and converts it into machine code line by line and executes it. This is why Java is platform independent: any machine that has a JVM can run the byte code.

****Features of Java:**** Simple, Object Oriented, Robust, Secured, Platform Independent, Portable, Multi-threaded.

****Types of Java Programs:****

Application: A stand-alone program that runs on a computer. It has a main() method.

Applet: A small Java program that runs within a web browser.

****3. Elementary Concepts of Objects and Classes****

****Class:**** A class is a user-defined blueprint or prototype from which objects are created. It represents a set of properties (data members) and methods that are common to all objects of that type. A class is a logical entity.

****Object:**** An object is an instance of a class. It has physical reality. Objects have state (attributes/instance variables) and behaviour (methods). For example, 'Dog' is a class, and a specific dog 'Tommy' is an object of that class.

Objects communicate with each other through message passing (calling methods).

****4. Values, Data Types and Tokens in Java****

****Tokens:**** The smallest individual units of a Java program. Types of tokens: Keywords, Identifiers, Literals, Punctuators (Separators), Operators.

****Keywords:**** Reserved words with a special meaning to the compiler. Examples: int, class, void, public, static, if, else, for.

****Identifiers:**** Names given to variables, methods, classes, etc.

****Data Types:****

Primitive Data Types:

byte (1 byte, -128 to 127)

short (2 bytes, -32768 to 32767)

int (4 bytes)

long (8 bytes)

float (4 bytes, decimal numbers)

double (8 bytes, larger decimal numbers)

char (2 bytes, Unicode, 0 to 65535)

boolean (not specified, true or false)

Non Primitive/Reference Data Types: Classes, Arrays, Interfaces.

****Type Conversion:****

Implicit (Widening) Conversion: Automatically done when a smaller type is assigned to a larger type. Example: int to long.

Explicit (Narrowing) Conversion: Manually done using type casting when a larger type is assigned to a smaller type. Example: double x = 9.5; int y = (int) x; // y becomes 9.

****5. Operators in Java****

****Arithmetic Operators:**** + (addition), - (subtraction), * (multiplication), / (division), % (modulus - remainder).

****Relational Operators:**** == (equal to), != (not equal to), >, <, >=, <=. They return a boolean result (true or false).

****Logical Operators:**** && (AND), || (OR), ! (NOT). Operate on boolean expressions.

****Assignment Operators:**** =, +=, -=, *=, /=, %=.

****Increment/Decrement Operators:**** ++ (adds 1), -- (subtracts 1). Prefix (++x) increments first then uses value. Postfix (x++) uses value first then increments.

****Ternary/Conditional Operator:**** condition ? expression1 : expression2. If condition is true, expression1 is evaluated; otherwise, expression2 is evaluated.

****new operator:**** Used to create new objects (dynamically allocates memory).

****dot (.) operator:**** Used to access members (variables and methods) of a class or object.

****Precedence and Associativity:**** Operators have a defined order of evaluation. When two operators have the same precedence, associativity determines the direction of evaluation.

****IMPORTANT QUESTIONS: JAVA PROGRAMMING BASICS****

****Q1.**** Explain the four principles of Object Oriented Programming with suitable real-life examples.

****Q2.**** Differentiate between Procedure Oriented Programming and Object Oriented Programming.

****Q3.**** Explain the compilation process in Java. What is byte code? What is the role of the JVM?

****Q4.**** Define class and object with an example. How does a class act as a blueprint for objects?

****Q5.**** State the difference between primitive data types and reference (composite) data types with examples.

****Q6.**** Explain implicit and explicit type conversion with an example of each.

****Q7.**** What are tokens in Java? Name the different types of tokens.

****Q8.**** State the difference between prefix and postfix increment operators with an example.

****Q9.**** Explain the different types of operators in Java with examples.

****Q10.**** State the use of the new operator and the dot (.) operator in Java.

****CHAPTER 2: INPUT MECHANISM IN JAVA****

****YouTube Lecture:**** https://youtu.be/06LT7JYS7Bo?si=xbJmHlfj9R3JQ_td

****1. Ways of Giving Input in Java****

****Initialisation:**** Data is given to the variable before the program runs (hardcoded). Example:
`int a = 10;`

****Parameters (Arguments):**** Data is passed to the method at the time of calling. Example:
`void display(int n)` – here `n` is the parameter.

****Input Stream (Scanner Class):**** Data is entered by the user at the time of program execution through the keyboard.

****2. The Scanner Class****

Scanner is a class present in the `java.util` package. It is used to take input from the user.

Steps to use Scanner:

Step 1: Import the Scanner class: `import java.util.Scanner;`

Step 2: Create a Scanner object: `Scanner sc = new Scanner(System.in);`

Step 3: Use methods of Scanner to read input:

`nextInt()` – reads an integer

`nextLong()` – reads a long

`nextFloat()` – reads a float

`nextDouble()` – reads a double

`nextShort()` – reads a short

`next()` – reads a single word (token) without space

`nextLine()` – reads the entire line including spaces

`next().charAt(0)` – reads a single character (takes the first character of the entered token)

****3. Types of Errors****

****Syntax Errors (Compile-time Errors):**** Occur when the programmer does not follow the grammatical rules (syntax) of the language. Example: missing semicolon, undeclared variable. The program will not compile.

****Runtime Errors:**** Occur during the execution of the program. The program compiles successfully but stops during execution. Example: division by zero, accessing an array element beyond its size.

****Logical Errors:**** The program compiles and runs, but gives wrong output because of incorrect logic. Example: using '+' instead of '*' for product calculation.

****4. Comments in Java****

****Single-line comment:**** Starts with `//`. Everything after `//` on the same line is ignored.

****Multi-line comment:**** Enclosed between `/*` and `*/`. Can span multiple lines.

****5. Output in Java****

`System.out.println()` : Prints the output and moves the cursor to the next line.

`System.out.print()` : Prints the output and the cursor stays on the same line.

****IMPORTANT QUESTIONS: INPUT MECHANISM IN JAVA****

****Q1.**** Explain the three ways of giving input in a Java program.

****Q2.**** What is the Scanner class? In which package is it present? Write the steps to use the Scanner class for reading a string and an integer.

****Q3.**** Write a Java statement using Scanner to read a single character from the user.

****Q4.**** Differentiate between the next() and nextLine() methods of the Scanner class.

****Q5.**** Explain the three types of errors that occur in a Java program with an example of each.

****Q6.**** What is the difference between a syntax error and a logical error?

****Q7.**** What is the purpose of comments in a Java program? Write the syntax for single-line and multi-line comments.

****Q8.**** What is the difference between System.out.println() and System.out.print()?

****CHAPTER 3: MATHEMATICAL LIBRARY METHODS****

****YouTube Lecture:**** <https://youtu.be/DWbooACEAqM?si=pv0luiTXiGNAQPC3>

****1. The Math Class****

The Math class is present in the java.lang package, which is the default package. It provides ready-made methods for performing basic mathematical operations. All methods of the Math class are static, meaning they are called using the class name: Math.methodName().

****2. Important Math Class Methods****

Math.pow(x, y): Returns x raised to the power y (x to the power y). Return type is double.

Example: Math.pow(2, 3) returns 8.0

Math.sqrt(x): Returns the positive square root of x. Return type is double.

Example: Math.sqrt(25) returns 5.0

Math.cbrt(x): Returns the cube root of x. Return type is double.

Example: Math.cbrt(27) returns 3.0

Math.ceil(x): Returns the smallest whole number greater than or equal to x (rounds up).
Return type is double.

Example: `Math.ceil(4.2)` returns 5.0; `Math.ceil(-4.2)` returns -4.0

`Math.floor(x)`: Returns the largest whole number less than or equal to x (rounds down). Return type is double.

Example: `Math.floor(4.8)` returns 4.0; `Math.floor(-4.8)` returns -5.0

`Math.round(x)`: Rounds x to the nearest whole number. Return type is long (for double) or int (for float).

Example: `Math.round(4.5)` returns 5; `Math.round(4.4)` returns 4

`Math.abs(a)`: Returns the absolute value (positive value) of a. Return type is same as argument type.

Example: `Math.abs(-10)` returns 10; `Math.abs(10)` returns 10

`Math.max(a, b)`: Returns the larger of the two values a and b.

Example: `Math.max(15, 25)` returns 25

`Math.min(a, b)`: Returns the smaller of the two values a and b.

Example: `Math.min(15, 25)` returns 15

`Math.random()`: Returns a double value between 0.0 (inclusive) and 1.0 (exclusive).

Example: `Math.random()` might return 0.7253...

To generate a random number between 0 and N: `(int)(Math.random() * (N+1))`

****3. Java Expressions****

An expression is a combination of variables, constants, operators, and method calls that evaluate to a single value. Use the Math class methods inside arithmetic expressions to calculate values.

****IMPORTANT QUESTIONS: MATHEMATICAL LIBRARY METHODS****

****Q1.**** What is the Math class? Which package contains it? Why are the methods of the Math class called using the class name?

****Q2.**** Write the return type and description for the following Math methods:

- (a) `sqrt(x)`
- (b) `cbrt(x)`
- (c) `pow(x, y)`
- (d) `ceil(x)`
- (e) `floor(x)`

****Q3.**** Differentiate between Math.ceil() and Math.floor() with examples.

****Q4.**** What is the output of Math.ceil(-5.3) and Math.floor(-5.3)? Give reasons.

****Q5.**** How can you generate a random number between 0 and 50 in Java?

****Q6.**** Write a Java expression for calculating the square root of x using Math.sqrt(). What should the value of x be to avoid an error?

****Q7.**** Evaluate the following expressions:

(a) Math.pow(3, 2) + Math.sqrt(36)

(b) Math.round(7.8) + Math.abs(-5)

****Q8.**** Write a Java statement to find the maximum of three numbers a, b, and c using Math methods.

****CHAPTER 4: IF ELSE AND NESTED IF ELSE (CONDITIONAL CONSTRUCTS)****

****YouTube Lecture:**** https://youtu.be/M8JYBz-R19o?si=eelNBHMSQkMoBi_Z

****1. Introduction to Conditional Constructs****

Conditional constructs (selection statements) are used to make decisions in a program. The program executes a set of instructions only if a certain condition is true. Java provides the following conditional constructs.

****2. The if Statement****

Syntax:

```
if (condition) {  
    // statements execute if condition is true  
}
```

If the condition is true, the body of the if block is executed. If false, nothing happens. Example: Checking if a number is positive.

****3. The if-else Statement****

Syntax:

```
if (condition) {  
    // statements execute if condition is true  
} else {  
    // statements execute if condition is false  
}
```

Used when one block must be executed when the condition is true, and another when it is false. Example: Check if a number is even or odd. If (number % 2 == 0), print "Even"; else print "Odd".

****4. The if-else-if Ladder (Multiple Conditions)****

Syntax:

```
if (condition1) {  
    // execute if condition1 true  
} else if (condition2) {  
    // execute if condition2 true  
} else if (condition3) {  
    // execute if condition3 true  
} else {  
    // execute if all conditions false  
}
```

Used when there are multiple conditions to check. Example: Grading system (if marks $\geq$ 90, A; else if marks $\geq$ 70, B; else if marks $\geq$ 50, C; else fail).

****5. Nested if****

Syntax:

```
if (outer condition) {  
    if (inner condition) {  
        // execute if both true  
    }  
}
```

An if statement inside another if statement. Used when a condition depends on another condition being true first.

****6. The switch-case Statement****

Syntax:

```
switch (expression) {  
    case value1:  
        // statements  
        break;  
    case value2:  
        // statements  
        break;  
    default:  
        // statements  
}
```

The expression is evaluated and compared against the case values. If a match is found, the corresponding block is executed. The break statement is used to exit the switch block. If no match is found, the default block is executed. *Fall through condition:* If break is not written after a case, execution falls through to the next case.

****7. System.exit(0)**** Terminates the current program. Used in menu-driven programs to end the program when the user chooses the exit option.

****IMPORTANT QUESTIONS: CONDITIONAL CONSTRUCTS****

****Q1.**** Explain the syntax of the if statement and the if-else statement with an example of each.

****Q2.**** Write a program using if-else-if ladder to check the grade of a student based on marks. Assume the grading system: ≥ 90 A, ≥ 70 B, ≥ 50 C, < 50 Fail.

****Q3.**** What is a nested if? Explain with an example program to find the largest of three numbers.

****Q4.**** What is the switch-case statement? How does it differ from the if-else-if ladder?

****Q5.**** Explain the fall-through condition in a switch-case statement with an example.

****Q6.**** What is the purpose of the break statement in a switch-case block? What happens if break is omitted?

****Q7.**** What is the role of the default case in a switch statement?

****Q8.**** What is the use of `System.exit(0)` in Java? When is it typically used?

****Q9.**** Write a menu-driven program that gives the user options to calculate the area of a circle, square, or rectangle based on their choice using switch-case.

****Q10.**** Write a program to check whether a given year is a leap year or not using nested if or logical operators.

****CHAPTER 5: LOOPS (ITERATIVE CONSTRUCTS)****

****YouTube Lecture:**** <https://youtu.be/ZGa33g5ii3k?si=QxJ9iDT94nyekAPd>

****1. Introduction to Loops****

Loops (iteration statements) are used to repeat a block of statements multiple times until a condition is met. Java provides three types of loops.

****2. for Loop (Entry Controlled Loop)****

Syntax:

```
for (initialisation; condition; update) {  
    // loop body
```

```
}
```

Initialisation sets the starting value (executed once). Condition is checked before each execution. If true, the body executes. After the body, the update statement runs, then condition is checked again. If false, the loop terminates. Used when the number of iterations is known in advance.

****3. while Loop (Entry Controlled Loop)****

Syntax:

```
while (condition) {  
    // loop body  
}
```

Condition is checked first. If true, the body executes and then condition is checked again. If false initially, the body never executes. Used when the number of iterations is not known in advance.

****4. do-while Loop (Exit Controlled Loop)****

Syntax:

```
do {  
    // loop body  
} while (condition);
```

Condition is checked after executing the loop body. The body is guaranteed to execute at least once even if the condition is false initially.

****5. Jump Statements****

****break:**** Terminates the loop immediately. Control passes to the statement after the loop.

****continue:**** Skips the current iteration and jumps to the next iteration of the loop.

****6. Inter-conversion of Loops****

Any for loop can be converted to a while loop and vice versa. The initialisation goes before the loop, the condition inside, and the update at the end of the loop body.

****IMPORTANT QUESTIONS: LOOPS (ITERATIVE CONSTRUCTS)****

****Q1.**** What is the difference between an entry-controlled loop and an exit-controlled loop? Give an example of each.

****Q2.**** Write the syntax of a for loop. Explain the purpose of the initialisation, condition, and update expressions.

****Q3.**** Write a program using a for loop to find the sum of the first N natural numbers.

****Q4.**** What is the difference between a while loop and a do-while loop? Give an example where a do-while loop is more appropriate.

****Q5.**** Explain the break and continue statements with an example of each.

****Q6.**** Write a program to reverse a number using a while loop. Input: 123, Output: 321.

****Q7.**** Write a program to check if a given number is a palindrome using a while loop.

****Q8.**** What is an infinite loop? How can it be prevented?

****Q9.**** Convert a given for loop into an equivalent while loop.

****Q10.**** Write a program to print the Fibonacci series up to N terms.

****CHAPTER 6: PATTERNS (NESTED FOR LOOPS)****

****YouTube Lecture:**** https://youtu.be/s33-gj4VHSM?si=JC8Q32JicT_xRGgJ

****1. Introduction to Nested Loops****

A nested loop means a loop inside another loop. For each iteration of the outer loop, the inner loop runs completely. Nested for loops are used to generate number and character patterns. The outer loop generally controls the number of rows, and the inner loop controls the elements in each row.

****2. Types of Patterns****

****Rectangular Pattern:**** Filled matrix of numbers or characters. Outer loop for rows, inner loop for columns. Print '*' or a number repeatedly.

****Right Angled Triangular Pattern:****

Increasing Triangle: Inner loop goes from 1 to current row number.

Decreasing Triangle: Inner loop goes from current row number to N.

****3. Approach to Pattern Problems****

Step 1: Identify the number of rows (outer loop runs from 1 to N).

Step 2: For each row, identify how many columns are needed (inner loop limit).

Step 3: Determine what to print (usually a number derived from row i, column j, or a constant character).

Step 4: Print a new line after each row using System.out.println().

****IMPORTANT QUESTIONS: PATTERNS (NESTED FOR LOOPS)****

****Q1.**** Write a program to print the following pattern (Right angled triangle of stars):

```
*  
* *  
* * *  
* * * *
```

****Q2.**** Write a program to print the following number pattern:

```
1  
1 2  
1 2 3  
1 2 3 4
```

****Q3.**** Write a program to print a filled square of asterisks (4 x 4 pattern).

****Q4.**** Write a program to print the following pattern:

```
1  
2 2  
3 3 3  
4 4 4 4
```

****Q5.**** Write a program to print the following inverted triangle pattern:

```
* * * *  
* * *  
* *  
*
```

****Q6.**** Explain what will be the output if the inner loop limit is changed from i to N in a right angled triangle pattern.

****Q7.**** What is the role of the outer loop and the inner loop in a nested for loop pattern program?

****Q8.**** Write a program to print a rectangular pattern of numbers 1 2 3 4 in all rows.

****CHAPTER 7: SERIES PROGRAMMING****

****YouTube Lecture:**** <https://youtu.be/vA1Vx5Qw5OE?si=eKa9gN5rkxsdSv6O>

****1. Introduction to Series****

A series is the summation or product of a sequence of terms following a specific rule. Programs on series require using loops to generate each term and accumulate a sum or product.

****2. Types of Series****

****Simple Arithmetic Series:**** $\text{Sum} = 1 + 2 + 3 + \dots + N$. Each term is directly obtained from the loop counter.

****Series based on Single Variable:**** $\text{Sum} = x + x^2 + x^3 + \dots + x^N$. Use `Math.pow(base, exponent)` to generate each term.

****Series with Factor in Denominator:**** $\text{Sum} = 1 + 1/2 + 1/3 + \dots + 1/N$. Be careful with type; use `1.0/i` to get a floating point result.

****Series with Alternating Signs (+ and -):**** Use a sign variable. For each term, multiply sign by `-1` to toggle signs.

****3. Approach to Series Problems****

Use a for loop from 1 to N.

Declare a variable `sum = 0` (for summation) or `prod = 1` (for product).

Inside the loop, compute the current term using the formula.

Add the term to sum (or multiply for product).

Print the sum after the loop ends.

****IMPORTANT QUESTIONS: SERIES PROGRAMMING****

****Q1.**** Write a program to find the sum of the series: $S = 1 + 2 + 3 + \dots + N$.

****Q2.**** Write a program to find the sum of the series: $S = 1 + 1/2 + 1/3 + \dots + 1/N$.

****Q3.**** Write a program to find the sum of the series: $S = 1 + x + x^2 + x^3 + \dots + x^N$.

****Q4.**** Write a program to find the sum of the series: $S = 1 - x + x^2 - x^3 + \dots$ N terms. (Use a sign variable to toggle +/-.)

****Q5.**** Write a program to find the product of the series: $P = 1 * 2 * 3 * \dots * N$.

****Q6.**** Write a program to find the sum of the series: $S = (1^2) + (2^2) + (3^2) + \dots + (N^2)$.

****Q7.**** What is the difference between summation and product of a series in terms of initialisation of the result variable?

****Q8.**** Write a program to find the sum of the series: $S = 1/a + 2/a^2 + 3/a^3 + \dots + N/a^N$.

****CHAPTER 8: USER-DEFINED METHODS (FUNCTIONS)****

****YouTube Lecture:**** <https://youtu.be/DWbooACEAqM?si=JYX1e1J13bM-Q8BW>

****1. Need for Methods****

A method is a block of code that performs a specific task and can be called (invoked) whenever needed. Methods promote reusability, modularity, and readability in a program. A complex problem can be broken down into smaller sub-problems, each handled by a method.

****2. Syntax of a Method****

```
<access specifier> <return type> <method name> (<parameter list>) {  
    // method body  
    [return value;] // if return type is not void  
}
```

****Access Specifier:**** public, private, protected. Controls visibility.

****Return Type:**** The data type of the value the method returns. Use void if it returns nothing.

****Method Name:**** Identifier used to call the method.

****Parameter List:**** Comma-separated list of variables that receive input values (formal parameters).

****3. Method Definition and Method Calling****

****Method Definition:**** Writing the method body (the code). A method is defined inside a class.

****Method Call/Invocation:**** Executing the method. A method is called using the object name (for non-static methods) or class name (for static methods).

Example: obj.methodName(arguments);

****4. Actual Parameters and Formal Parameters****

****Formal Parameters:**** The variables defined in the method signature (receiving parameters).

****Actual Parameters (Arguments):**** The values passed to the method when it is called.

****5. Call by Value (Pass by Value)****

In Java, parameters are passed by value. This means a copy of the actual parameter's value is made and passed to the formal parameter. Any changes made to the formal parameter inside the method do not affect the original actual parameter.

****6. Pure and Impure Methods****

****Pure Method:**** A method that does not change the state (instance variables) of the object. It only returns a value based on the input parameters. Example: Math.sqrt(), any method that just computes and returns a value.

****Impure Method:**** A method that changes the state of the object (modifies instance variables). Example: A setter method that updates an instance variable.

****7. Static and Non-Static Methods****

****Static Method:**** Declared using the keyword 'static'. Can be called without creating an object. Uses the class name for calling. Cannot access non-static instance variables directly unless through an object.

****Non-Static Method:**** Not declared with 'static'. Must be called through an object of the class. Can directly access instance variables.

****8. Method Overloading (Polymorphism)****

Method overloading is when two or more methods in the same class have the same name but different parameter lists (different number of parameters, different data types of parameters, or both). The return type alone cannot distinguish overloaded methods.

****IMPORTANT QUESTIONS: USER-DEFINED METHODS (FUNCTIONS)****

****Q1.**** What is a method? Explain the need for methods with an example.

****Q2.**** Explain the difference between actual parameters and formal parameters.

****Q3.**** What is meant by call by value? Explain with an example and show that the original variable is not affected.

****Q4.**** Differentiate between pure and impure methods with an example of each.

****Q5.**** What is the difference between a static method and a non-static method? How is each one called?

****Q6.**** What is method overloading? How does it demonstrate polymorphism? Give an example of overloaded methods to find the area of a circle (radius given) and area of a rectangle (length and breadth given).

****Q7.**** Write a class with a method that takes two numbers and returns their sum. Is this method pure or impure? Justify.

****Q8.**** Explain the method prototype (signature) with an example. What constitutes the method prototype?

****Q9.**** Can overloaded methods have different return types alone? Explain with reason.

****Q10.**** Write a program with a method that accepts a number and returns the reverse of the number.

****CHAPTER 9: CONSTRUCTORS****

****YouTube Lecture:**** <https://youtu.be/5pm61kW5hYs?si=8PQj4VBWgxzUwVdl>

****1. Definition of a Constructor****

A constructor is a special type of method that is automatically called when an object of a class is created. It has the same name as the class. Its main purpose is to initialise the instance variables of the object.

****2. Characteristics of a Constructor****

It has the same name as the class.

It has no return type, not even void.

It is automatically invoked when the object is created using the new keyword.

If no constructor is defined, the compiler provides a default constructor (no parameters, empty body).

A class can have more than one constructor (constructor overloading).

****3. Types of Constructors****

****Default Constructor:**** A constructor that takes no parameters. If not written, the compiler provides it. If you write any constructor, the compiler does not provide the default constructor.

Syntax: `ClassName() { // body }`

****Parameterized Constructor:**** A constructor that accepts parameters. It is used to initialise object properties with specific values at the time of creation.

Syntax: `ClassName(int a, String b) { // body }`

****4. Constructor Overloading****

Constructors with the same name (class name) but different parameter lists. This allows an object to be created in different ways by passing different arguments.

Example:

`Employee() { ... } // default`

`Employee(String name) { ... }`

`Employee(String name, int id) { ... }`

****5. Difference Between Constructor and Method****

Constructor: Has the same name as the class. No return type. Automatically called when object is created. Used to initialise objects.

Method: Can have any valid name. Must have a return type. Called explicitly using the object. Used to define the behaviour/functionality of the object.

****IMPORTANT QUESTIONS: CONSTRUCTORS****

****Q1.**** What is a constructor? State any three characteristics of a constructor.

****Q2.**** Explain the difference between a default constructor and a parameterized constructor.

****Q3.**** When is a default constructor automatically provided by the compiler? When is it not provided?

****Q4.**** What is constructor overloading? Give an example of a class with an overloaded constructor.

****Q5.**** Write a class Student with instance variables name and rollNumber. Write a default and a parameterized constructor to initialise these variables.

****Q6.**** Differentiate between a constructor and a method (any three points).

****Q7.**** What happens if you write only a parameterized constructor and try to create an object using the default constructor? Explain with reason.

****Q8.**** Explain the role of the new operator in relation to a constructor.

****Q9.**** Can a constructor be private? If yes, can an object of that class be created? Explain.

****CHAPTER 10: LIBRARY CLASSES (WRAPPER CLASSES)****

****YouTube Lecture:**** <https://youtu.be/DWbooACEAqM?si=JYX1e1J13bM-Q8BW> (Also covers wrapper class concepts along with methods.)

****1. Introduction to Wrapper Classes****

A wrapper class wraps (encloses) a primitive data type into an object. All primitive data types in Java have a corresponding wrapper class.

byte -> Byte
short -> Short
int -> Integer
long -> Long
float -> Float
double -> Double
char -> Character
boolean -> Boolean

****2. Autoboxing and Unboxing****

****Autoboxing:**** The automatic conversion of a primitive data type into its corresponding wrapper class object.

Example: Integer i = 5; // int to Integer automatically

****Unboxing:**** The automatic conversion of a wrapper class object back into its corresponding primitive data type.

Example: `int a = i;` // Integer to int automatically

****3. Important Methods of Wrapper Classes****

****Integer.parseInt(String s):**** Converts a numeric String s into an int primitive.

Example: `int a = Integer.parseInt("123");` // a = 123

****Long.parseLong(String s):**** Converts a numeric String s into a long.

****Float.parseFloat(String s):**** Converts a numeric String s into a float.

****Double.parseDouble(String s):**** Converts a numeric String s into a double.

****Character.isDigit(char ch):**** Returns true if ch is a digit (0-9).

****Character.isLetter(char ch):**** Returns true if ch is a letter (A-Z, a-z).

****Character.isLetterOrDigit(char ch):**** Returns true if ch is a letter OR digit.

****Character.isLowerCase(char ch):**** Returns true if ch is lowercase.

****Character.isUpperCase(char ch):**** Returns true if ch is uppercase.

****Character.isWhitespace(char ch):**** Returns true if ch is a space, tab, or newline.

****Character.toLowerCase(char ch):**** Returns the lowercase equivalent of ch.

****Character.toUpperCase(char ch):**** Returns the uppercase equivalent of ch.

****IMPORTANT QUESTIONS: LIBRARY CLASSES (WRAPPER CLASSES)****

****Q1.**** What is a wrapper class? Name the wrapper class for each primitive data type.

****Q2.**** Explain autoboxing and unboxing with an example of each.

****Q3.**** Write a statement using `Integer.parseInt()` to convert a string "456" into an int. What happens if the string is "abc"?

****Q4.**** State the purpose of the following methods: `parseFloat()`, `parseLong()`.

****Q5.**** What is the difference between `Character.isDigit()` and `Character.isLetter()`? Write an example showing their use.

****Q6.**** Write a Java segment to check if a character is uppercase and, if it is, convert it to lowercase. Print the result.

****Q7.**** What does `Character.isWhitespace()` check? Give an example where it can be used.

****Q8.**** Differentiate between a primitive data type and a composite data type. Give an example of each.

****CHAPTER 11: STRING HANDLING****

****YouTube Lecture:**** https://youtu.be/oR81oaKdrhk?si=SW_BpaPN3Jxn84Pw

****1. Introduction to the String Class****

String is a built-in class in Java present in the java.lang package. A string is a sequence of characters. Strings in Java are immutable, meaning once created, their value cannot be changed. Any operation that appears to modify a string actually creates a new string object.

Creating a string: String s = "Hello"; or String s = new String("Hello");

****2. Important String Methods****

****int length():**** Returns the number of characters in the string (including spaces).

Example: "Hello".length() returns 5

****char charAt(int index):**** Returns the character at the specified index position. Index starts from 0.

Example: "Hello".charAt(1) returns 'e'

****int indexOf(char ch):**** Returns the index of the first occurrence of character ch. Returns -1 if not found.

Example: "Hello".indexOf('l') returns 2

****int lastIndexOf(char ch):**** Returns the index of the last occurrence of character ch.

Example: "Hello".lastIndexOf('l') returns 3

****String toLowerCase():**** Returns a new string with all characters converted to lowercase.

****String toUpperCase():**** Returns a new string with all characters converted to uppercase.

****String trim():**** Returns a new string with leading and trailing whitespace removed.

****String concat(String str):**** Returns a new string by joining the given string str to the end.

Example: "Hello".concat(" World") returns "Hello World". Same as + operator.

****boolean equals(String str):**** Returns true if the two strings are exactly equal (case sensitive).

****boolean equalsIgnoreCase(String str):**** Returns true if two strings are equal ignoring case differences.

****int compareTo(String str):**** Compares two strings lexicographically (alphabetically/dictionary order). Returns 0 if equal, a positive value if the calling string is lexicographically greater, negative if smaller.

****int compareToIgnoreCase(String str):**** Same as compareTo but ignores case.

****String replace(char oldChar, char newChar):**** Returns a new string with all occurrences of oldChar replaced by newChar.

****String substring(int beginIndex):**** Returns the substring from beginIndex to the end.

****String substring(int beginIndex, int endIndex):**** Returns the substring from beginIndex (inclusive) to endIndex (exclusive).

****boolean startsWith(String prefix):**** Returns true if the string starts with the given prefix.

****boolean endsWith(String suffix):**** Returns true if the string ends with the given suffix.

****static String valueOf(any primitive type):**** Converts a primitive value to its String representation.

****IMPORTANT QUESTIONS: STRING HANDLING****

****Q1.**** What is the String class? What does it mean that strings in Java are immutable?

****Q2.**** Explain the difference between equals() and compareTo() methods. Write a program segment to check if two strings are equal ignoring case.

****Q3.**** Differentiate between the following methods with examples:

- (a) indexOf() and lastIndexOf()
- (b) length() and charAt()
- (c) toLowerCase() and toUpperCase()

****Q4.**** What is the output of the following?

- (a) " Computer ".trim().length()
- (b) "Hello".replace('l', 'w')

****Q5.**** Write a program to input a string and print it in reverse order using charAt() and length() methods.

****Q6.**** Write a program to check whether a given string is a palindrome using string methods.

****Q7.**** Explain the substring() method. Write a program to extract the first 3 characters and the last 3 characters of a given string.

****Q8.**** Write a program to input a sentence and count the number of vowels and consonants using string methods and Character wrapper class methods.

****Q9.**** What does the startsWith() and endsWith() method do? Give an example.

****Q10.**** What does the valueOf() method do? How is it different from the parse methods?

****CHAPTER 12: ARRAYS (SINGLE DIMENSIONAL)****

****YouTube Lecture:**** https://youtu.be/N5FK6JtkAkA?si=W4Wy_R4SOw_svvk5

****1. Introduction to Arrays****

An array is a collection of variables of the same data type, stored in contiguous memory locations, and accessed using a common name with an index. It is a composite (reference) data type. The length statement (arrayName.length) gives the number of elements.

****2. Declaration, Initialisation and Accessing Arrays****

****Declaration:**** dataType arrayName[]; OR dataType[] arrayName;

Example: int arr[];

****Allocation:**** arrayName = new dataType[size];

Example: arr = new int[5];

****Declaration and Allocation Together:**** int arr[] = new int[5];

****Initialisation:**** Assigning values.

At declaration: int arr[] = {5, 10, 15, 20, 25};

Using loop: for(int i = 0; i < arr.length; i++) arr[i] = sc.nextInt();

****Accessing Elements:**** arrayName[index] – index starts from 0 and goes up to (length - 1).

****3. Sorting Techniques (Arranging in Ascending/Descending Order)****

****A. Selection Sort****

Find the smallest element in the unsorted part and swap it with the first element of the unsorted part. Repeat until sorted.

Outer loop: i from 0 to (n-2)

Inner loop: j from (i+1) to (n-1)

Inside: if arr[j] < arr[pos_min], update pos_min.

After inner loop: swap arr[i] and arr[pos_min].

****B. Bubble Sort****

Repeatedly compare adjacent elements and swap if they are in the wrong order. In each pass, the largest element "bubbles" to its correct position at the end.

Outer loop: i from 0 to (n-2)

Inner loop: j from 0 to (n-2-i)

Inside: if arr[j] > arr[j+1], swap them.

****4. Search Techniques****

****A. Linear Search****

Searches the element sequentially from the start to the end. Used when the array is unsorted. If the element is found, return its index; else, return -1. Simple but can be slow for large arrays.

****B. Binary Search****

Searches the element in a sorted array. Works by repeatedly dividing the search interval in half.

Find low (0), high (length - 1), $mid = (low + high) / 2$.

If $target == arr[mid]$, element found.

If $target < arr[mid]$, search in the left half: $high = mid - 1$.

If $target > arr[mid]$, search in the right half: $low = mid + 1$.

Repeat while $low \leq high$.

Binary search is much faster than linear search for large sorted arrays.

****IMPORTANT QUESTIONS: ARRAYS (SINGLE DIMENSIONAL)****

****Q1.**** Define an array. How is an array declared, allocated memory, and initialised? Write the syntax for each.

****Q2.**** Write a program to input 10 numbers into an array and find the sum of all elements.

****Q3.**** Explain the selection sort technique. Write a program to sort an array in ascending order using selection sort.

****Q4.**** Explain the bubble sort technique. Write a program to sort an array in ascending order using bubble sort.

****Q5.**** Write a program to search for a given element in an array using linear search. Print the index if found.

****Q6.**** Explain the binary search technique. Write a program to search for a given element in a sorted array using binary search.

****Q7.**** What is the difference between linear search and binary search? When is each suitable?

****Q8.**** Write a program to find the largest and smallest element in an array.

****Q9.**** What does `arr.length` return? Give an example of its use in a for loop.

****Q10.**** What happens if you try to access an array element at an index equal to the length of the array? Explain with reason.

****CHAPTER 13: 2-D ARRAY (DOUBLE DIMENSIONAL ARRAY)****

****YouTube Lecture:**** https://youtu.be/ZJ-Cb_zRxul?si=j1-CXFZMsFDc9SQ3

****1. Introduction to Double Dimensional Arrays****

A double dimensional (2-D) array is an array of arrays. It has rows and columns, forming a matrix or table. It is a composite data type.

****2. Declaration, Initialisation and Accessing****

****Declaration:**** dataType arr[][]; OR dataType[][] arr;

****Allocation:**** arr = new dataType[rows][columns];

Example: int mat[][] = new int[3][3]; // 3x3 matrix

****Declaration and Allocation Together:**** int mat[][] = new int[3][4];

****Initialisation:**** Use nested loops. Outer loop for rows (i), inner loop for columns (j). mat[i][j] = sc.nextInt();

****Accessing Elements:**** mat[row_index][col_index]

****3. Displaying a Double Dimensional Array in Matrix Format****

Use nested for loops to print each element. After the inner loop completes (each row), print a new line using System.out.println() to move to the next row.

****4. Operations on 2-D Arrays****

****Sum of Row Elements:**** Outer loop for rows. Sum variable = 0. Inner loop for columns adds mat[i][j] to sum. Print sum for each row.

****Sum of Column Elements:**** Outer loop for columns. Sum = 0. Inner loop for rows adds mat[i][j] to sum. Print sum for each column.

****Sum of Diagonal Elements:****

Left Diagonal (Principal Diagonal): Elements where i == j. mat[0][0], mat[1][1], mat[2][2].

Right Diagonal: Elements where i + j == N - 1. mat[0][2], mat[1][1], mat[2][0] for a 3x3 matrix.

****IMPORTANT QUESTIONS: 2-D ARRAY (DOUBLE DIMENSIONAL ARRAY)****

****Q1.**** Define a double dimensional array. Write the syntax for declaring and allocating a 2-D array of size 4x4.

****Q2.**** Write a program to input elements into a 3x3 double dimensional array and display it in matrix format.

****Q3.**** Write a program to find the sum of the elements of each row in a 2-D integer array.

****Q4.**** Write a program to find the sum of the elements of each column in a 2-D integer array.

****Q5.**** Write a program to find the sum of the left diagonal and right diagonal elements of a square matrix.

****Q6.**** Write a program to find the maximum element in a 2-D array.

****Q7.**** How many elements are there in a double dimensional array declared as `int a[][] = new int[3][4]`? What is the valid range of row and column indices?

****Q8.**** Write a program to transpose a square matrix (swap rows and columns).
